claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a67423b79c2bcd0b8.jsonl`
- SHA-256: `0dd1574d45b6cf493f48c0bef028c12858f6c85f7235f64ffa4f3cfc6f71c962`
- Source modified: `2026-05-31T04:18:52+00:00`
- Imported at: `2026-07-05T16:48:20+00:00`
- project: `wf_076beba8-59c`
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`

Transcript

1. user (2026-05-31T04:17:40.132Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):
1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:
- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON

transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:
URL: https://github.com/microsoft/table-transformer
Title: microsoft/table-transformer (TATR) ? official repo, PubTables-1M dataset & GriTS metric
Found via: OCR + table-structure recognition pipelines (practitioner/implementation) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
   - be a concrete, checkable statement (not vague generalities)
   - include a direct quote from the source as support
   - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:17:42.671Z)

I'll fetch the GitHub page for microsoft/table-transformer and extract key claims relevant to the research question.

3. assistant (2026-05-31T04:17:49.335Z)

I have the WebFetch tool loaded. Now let me fetch the GitHub page for microsoft/table-transformer.

4. user (2026-05-31T04:17:59.318Z)

Table Transformer (TATR) - Key Details

(1) Code License

The repository uses an **MIT license**. This is stated in the LICENSE file reference shown in the navigation.

(2) Model Weights & Dataset Licensing

The webpage does not explicitly state the licenses for the pre-trained model weights or the PubTables-1M dataset. The models are hosted on Hugging Face, but specific licensing terms are

not detailed in this content.

(3) What TATR Does

"A deep learning model based on object detection for extracting tables from PDFs and images."
The system performs two primary tasks:
- **Table Detection**: Identifying tables in document pages
- **Table Structure Recognition**: Understanding cell topology, bounding boxes, and text content within tables

(4) GriTS Metric

GriTS measures "table cell correctness" by evaluating predicted cells based on: "(1) cell topology alone, (2) cell topology and the reported bounding box location of each cell, or (3) cell topology and the reported text content."

(5) Release & Maintenance Status

- **Latest Release**: v1.0.0 (February 17, 2023)
- **Recent Updates**: Most recent news entry is August 22, 2023, announcing new pre-trained models (TATR-v1.1)
- The repository shows 195 commits with active development

(6) Python Integration & Hugging Face

The project provides code for installation via conda and includes "a new simple inference pipeline" released March 7, 2023. PubTables-1M is "alternatively available for download from Hugging Face."

(7) Model Architecture

The system uses **DETR (Detection Transformer)** architecture. Specifically: "DETR R18" backbone for both detection and structure recognition models.

(8) Performance Claims

Table Detection (DETR R18 on PubTables-1M):
- AP50: 0.995, AP75: 0.989, AP: 0.970

Table Structure Recognition (TATR-v1.0):
- AP: 0.902, GriTS metrics including topology correctness of 0.9849

5. assistant (2026-05-31T04:18:02.574Z)

The WebFetch gave good detail but did not specify the weights/dataset license, which is critical for the research question. Let me check the README directly and the model weights license, since the question specifically demands verification of the WEIGHTS and PubTables-1M dataset license.

6. user (2026-05-31T04:18:14.036Z)

License Information & Key Details

Model Weights/Pretrained Checkpoints

The webpage does not provide explicit license statements for the model weights or pretrained checkpoints. It only indicates that models are available on Hugging Face model cards, but no specific license text is quoted on this page.

Dataset Licenses

Similarly, the page does not contain quoted license statements (including CDLA, MIT, or commercial use restrictions) for PubTables-1M, FinTabNet, or other datasets mentioned.

What TATR Does

"A deep learning model based on object detection for extracting tables from PDFs and images."

GriTS

"Grid table similarity metric for table structure recognition" - described as measuring table cell correctness by evaluating cell topology, bounding box location, and text content accuracy.

Latest News & Hugging Face Availability

- **Most recent update (08/22/2023):** "We have released 3 new pre-trained models for TATR-v1.1 (trained on 1. PubTables-1M, 2. FinTabNet.c, and 3. both datasets combined)"
- **Hugging Face:** The full PubTables-1M dataset is "alternatively available for download from Hugging Face"
- Multiple pre-trained model weights are hosted on Hugging Face with dedicated model cards

Note: For definitive licensing information, you would need to consult the actual Hugging Face model cards or dataset repositories directly.

7. assistant (2026-05-31T04:18:19.080Z)

The README itself does not state the weights/dataset license inline. The research question explicitly flags this as critical ("Verify the WEIGHTS license, not just the repo license"). Let me check the Hugging Face dataset card for PubTables-1M and the model card, since that is where the license is authoritatively stated. I'll fetch both in parallel.

8. user (2026-05-31T04:18:23.584Z)

License of PubTables-1M Dataset

The license of the PubTables-1M dataset is **CDLA-Permissive-2.0** (Community Data License Agreement - Permissive 2.0).

This is explicitly stated in the dataset page header:

License: cdla-permissive-2.0

The CDLA-Permissive-2.0 is a permissive license that allows broad use of the dataset for various purposes while maintaining appropriate attribution requirements.

9. user (2026-05-31T04:18:25.139Z)

Table Transformer Structure Recognition Model

License: MIT

Model Description:
The Table Transformer is a DETR (Detection Transformer)-based object detection model fine-tuned for table structure recognition. It was trained on PubTables1M and introduced in the paper "PubTables-1M: Towards Comprehensive Table Extraction From Unstructured Documents" by Smock et al.

Architecture:
- Transformer-based object detection model (equivalent to DETR)
- Uses the "normalize before" setting, meaning layernorm is applied before self- and cross-attention
- Model size: 28.8M parameters

What it does:
The model detects the structure of tables in images, identifying elements like rows and columns. It's useful for table extraction from unstructured documents like PDFs.

Example Usage:
```python
from transformers import pipeline

pipe = pipeline("object-detection", model="microsoft/table-transformer-structure-recognition")
```

Or with direct model loading:
```python
from transformers import AutoImageProcessor, AutoModelForObjectDetection

processor = AutoImageProcessor.from_pretrained("microsoft/table-transformer-structure-recognition")
model = AutoModelForObjectDetection.from_pretrained("microsoft/table-transformer-structure-recognition")
```

10. assistant (2026-05-31T04:18:32.733Z)

I now have authoritative license information for all three components (code, weights, dataset). This is exactly what the research question needs. Let me compile the structured output.

Summary of findings, with the key license facts verified across the canonical sources:
- **Code:** MIT (repo LICENSE) ? verified.
- **Model weights** (`microsoft/table-transformer-structure-recognition` HF card): **MIT** ? verified, commercially safe.
- **PubTables-1M dataset** (`bsmock/pubtables-1m` HF card): **CDLA-Permissive-2.0** ? a permissive, commercially-usable data license (this matters because the research question warned that many doc-AI training datasets are non-commercial; TATR's is not).
- Architecture: DETR-based object detection (28.8M params, R18 backbone), small VRAM footprint.
- Maintenance: last release v1.0.0 (Feb 2023), last news Aug 2023 (TATR-v1.1 trained on PubTables-1M + FinTabNet.c) ? relatively stale.
- Python: first-class via Hugging Face `transformers` (`AutoModelForObjectDetection`).
- GriTS metric and performance claims captured.

This is a primary source (official Microsoft Research repo + the original authors' HF cards/dataset).

11. user (2026-05-31T04:18:43.389Z)

Structured output provided successfully

12. assistant (2026-05-31T04:18:52.714Z)

I've completed the source extraction. Here's a summary of what I found and verified for the microsoft/table-transformer (TATR) source:

Source quality: primary ? the official Microsoft Research GitHub repo plus the original authors' Hugging Face model/dataset cards.

Key verified findings (the licensing question the research brief flagged as critical):

1. **Code license: MIT** (repo LICENSE)
2. **Model weights license: MIT** ? verified on the `microsoft/table-transformer-structure-recognition` HF card. Commercially safe.
3. **PubTables-1M dataset license: CDLA-Permissive-2.0** ? verified on the `bsmock/pubtables-1m` HF card. This is a permissive, commercially-usable data license. Important contrast: the research brief warned that many doc-AI training datasets are non-commercial (e.g. LayoutLMv3's), but TATR's training data is **not** restricted.

Other extracted facts relevant to muneem:
- **Architecture:** DETR-based object detection, **28.8M params**, R18 backbone ? small enough for the lowest (~8-12 GB) consumer VRAM tier.
- **Function:** table detection + table structure recognition (rows/columns/cell topology) ? a table-structure specialist, not an end-to-end OCR engine. Relevant to the brief's borderless/semi-ruled table question (you'd pair it with a text OCR layer).
- **Python path:** first-class via HF `transformers` (`AutoModelForObjectDetection`), no GPL entanglement.
- **Maintenance:** largely stale ? last release v1.0.0 (Feb 2023), last news Aug 2023 (TATR-v1.1 on PubTables-1M + FinTabNet.c). No recent active development as of 2025-2026.
- GriTS metric and AP performance numbers also captured in the WebFetch output.

The structured output has been submitted with 5 falsifiable claims (2 central, 3 supporting) and publish date 2023-08-22.